



ASD2024 - II Appello - Teoria

Indietro

Domanda 1

Punteggio max.:
1,50

Contrassegna
domanda

Sia dato il seguente insieme di attività caratterizzate da tempo di inizio e tempo di fine $[s, f)$:

$[0,7)$ $[1,5)$ $[6,11)$ $[12,15)$ $[8,13)$ $[8,12)$ $[3,9)$ $[3,6)$ $[2,14)$ $[5,10)$ $[5,8)$

Si determini mediante un algoritmo greedy l'insieme con il massimo numero di attività compatibili.

↓

A ▾

B

I

✎ ▾

☰

☷

☰

☷

🔗

🔄

😊

🖼️

👤

🔍

Formato della risposta:

- quale intervallo è stato considerato al passo $i=4$? $5,8$
- l'intervallo $[8,13)$ fa parte della soluzione? sì/no no
- insieme contenente il massimo numero di attività compatibili

Massimo numero di attività mutuamente compatibili

1. Ordinare le attività in ordine di tempi di fine crescenti.
2. Prendere la prima attività, e confrontarla con tutte le altre.
 - a. Se si intersecano, allora la seconda attività non si può prendere;
 - b. Se non si intersecano, allora la seconda attività si può prendere.
3. La soluzione è definita mentre si percorrono le attività.

1) $0, 10, 1, 0$

$(1,5)$ $(3,6)$ $(0,7)$ $(5,8)$ $(3,9)$ $(5,10)$ $(6,11)$ $(8,12)$ $(8,13)$ $(2,14)$ $(12,15)$

i

1	1,5	✓
2	3,6	X
3	0,7	X
4	5,8	✓

$\{ [1,5) [5,8) [8,12) [12,15) \}$

...

Domanda 2

Punteggio max.:
2,50

Contrassegna
domanda

Si determini mediante un algoritmo di programmazione dinamica una Longest Increasing Sequence della sequenza

8, 5, 7, 4, 9, 10, 3, 11, 6

Formato della risposta:

per i vettori L e P e per la LIS finale: elenco su una sola riga degli elementi separati da virgole o spazi:

$el1, el2, el3, \dots, eln$

Contenuto dei vettori L e P:

al passo $i=4$

L: 1 1 2 1 3 1 1 1 1

P: -1 -1 1 -1 2 -1 -1 -1 -1

al passo $i=6$:

L: 1 1 2 1 3 5 1 1 1

P: -1 -1 1 -1 2 4 -1 -1 -1

LIS: 5 7 9 10 11

Longest Increasing Sequence

- Definire una tabella con 3 righe: la prima contiene una sequenza, la seconda contiene la lunghezza L della sotto-sequenza e la terza contiene l'indice del valore precedente P nella sotto-sequenza.
 - I valori della lunghezza sono inizializzati a 1.
 - I valori del precedente sono inizializzati a -1.
- Il primo valore nella sequenza viene inizializzato con valori $L = 1, P = -1$.
- Dal secondo valore in poi, si controllano tutti i valori precedenti nella sequenza.
 - Se il valore è minore dell'elemento con cui si sta confrontando, allora si passa al successivo.
 - Se la lunghezza attuale è minore della lunghezza di uno qualsiasi degli elementi, allora si cambia il valore di L con la lunghezza dell'altro elemento incrementato di 1 e si cambia il valore di P con l'indice dell'elemento con cui si è confrontata la lunghezza.
- Ripetere il punto 3 fino al termine della sequenza.
- Per stampare la soluzione, a partire dall'elemento con valore L massimo, si stampa la sequenza relativa, definita dai predecessori di quell'elemento.

i	0	1	2	3	4	5	6	7	8	
A	8	5	7	4	9	10	3	11	6	
L		1	1	2	1	3	4	1	5	2
P		-1	-1	1	-1	2	4	-1	5	1

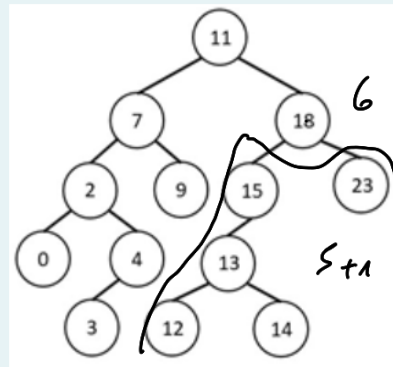
sotto seq. (5-6)

Domanda 3

Punteggio max.:
1,00

Contrassegna
domanda

Sia dato il seguente BST:



Lo si estenda, aggiungendo in ogni nodo le informazioni necessarie alla funzione BSTselect.

↴

A ▾

B

I

✎ ▾

☰

☰
1
2
3

☰
1
2
3

☰
1
2
3

☰
1
2
3

🔗

🔄

😊

🖼️

👤

🔍

Formato della risposta:

Informazioni aggiunte al nodo 18: 6

Archi percorsi per raggiungere il nodo con chiave di rango 4:

arco 1:

...

arco n:

BST SELECT VUOLE SIZE DI OGNI NODO

• CLICHIAMO $K: 4$

1) $11 \mid 7$
 ↑
 $4 \leq 7$
SINISTRA

→ $7 - 2$
 ↑
 $4 \leq 5$
SINISTRA

→ $2 - 1$
 ↑
 $4 > 2$
DESTRA

→ $2 = 2$
 Ⓢ
AGGIORNAMENTO
 $K_{NUO} = K - 1$
 $4 - 2 = 2$

Domanda 4

Punteggio max.:
2,00

Contrassegna
domanda

Sia data la sequenza di interi, supposta memorizzata in un vettore:

10 5 9 34 6 70 14 21 11 44 1 8 12 13

si eseguano i primi 2 passi dell'algoritmo di quicksort per ottenere un ordinamento ascendente, indicando ogni volta il pivot scelto. NB: i passi sono da intendersi, impropriamente, come in ampiezza sull'albero della ricorsione, non in profondità.

↓

A ▾

B

I

Formato della risposta: riportare le 2 partizioni del vettore originale e le due partizioni delle partizioni trovate al punto precedente.

Formato:

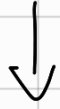
- partizione del vettore originale:
|sottovettSX | pivot | sottovettDX|
- partizione del sottovettore sinistro:
|sottovettSX1| pivot1 |sottovettDX1|
- partizione del sottovettore destro:
|sottovettSX2|pivot2|sottovettDX2|

QuickSort

1. Definisci il pivot come l'elemento all'ultimo indice del vettore.
2. Definisci l'indice i come quello del primo elemento del vettore restante, e l'indice j come quello dell'ultimo elemento del vettore restante.
 - a. Scorri l'indice i a destra fino a trovare un elemento maggiore del pivot.
 - b. Scorri l'indice j a sinistra fino a trovare un elemento minore del pivot.
3. Confrontiamo gli indici.
 - a. Se $i \geq j$, passa al punto successivo.
 - b. Se $i < j$ scambia gli elementi all'indice i e all'indice j e ripeti dal punto 2.
4. Scambia l'elemento all'indice i con quello all'indice del pivot.
5. Utilizzando l'elemento pivot come divisore, dividi in due sotto-vettori il vettore principale, ed esegui il QuickSort su entrambi.
6. Termina quando il vettore è ordinato.

10 5 9 34 6 70 14 21 11 44 1 8 12 13 $\rightarrow$ pivot

10 5 9 12 6 8 1 (11) | 13 | 44 14 70 34 (21)
pivot sk pivot sk



10 5 9 1 6 8 | 11 | 12

14 | 21 | 70 34 44

sk

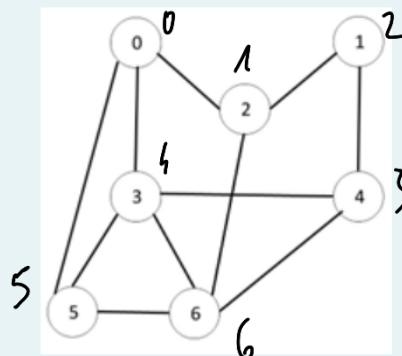
sk

Domanda 5

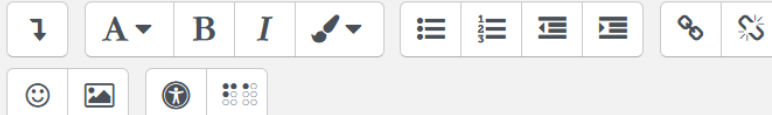
Punteggio max.:
1,00

Contrassegna
domanda

Si determinino i punti di articolazione del seguente grafo non orientato:



Si consideri **0** come vertice di partenza e, qualora necessario, si trattino i vertici secondo l'ordine numerico e si assuma che la lista delle adiacenze sia anch'essa ordinata numericamente.



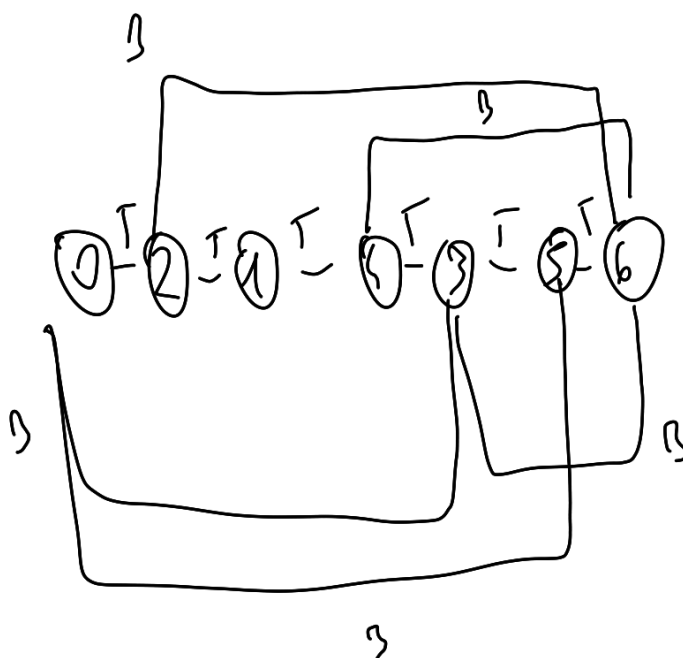
Formato:

- elenco vertici che sono punti di articolazione $\rightarrow$ *7 1 2 3 4 5 6*
- elenco degli archi backward

Punti di articolazione

1. Si applica la visita in profondità al grafo, ottenendo l'albero di visita in profondità con i relativi archi T, B, F, C.
2. I punti di articolazione sono tutti i nodi che mantengono connesso il grafo, anche definiti come nodi nella quale i nodi figli non hanno archi di tipo B che connettono ad un antenato del nodo candidato.

B: (3,0) (5,0) (6,3) (6,2)
(6,4)

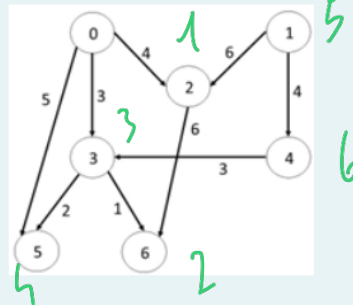


Domanda 6

Punteggio max.: 2,00

Contrassegna domanda

Si determinino per il seguente DAG pesato i valori di tutti i cammini MASSIMI che collegano il vertice 0 con ogni altro vertice: 0



Si trattino, qualora necessario, i vertici secondo l'ordine numerico e si assuma che la lista delle adiacenze sia anch'essa ordinata numericamente.

Formato: stime della distanza massima ad ogni passo

Passo 0
0: 1: 2: 3: 4: 5: 6:

Passo 1
0: 1: 2: 3: 4: 5: 6:

....

Passo n
0: 1: 2: 3: 4: 5: 6:

Cammini massimi del DAG

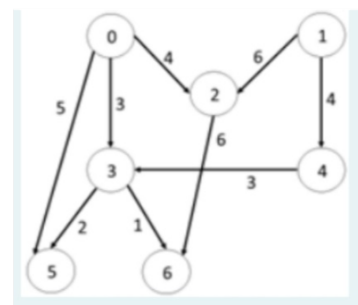
- Si effettua l'ordinamento topologico del DAG dal nodo x.
- Inizialmente si impostano i cammini massimi dal nodo x a tutti gli altri nodi come $-\infty$, mentre quello da cui si parte a 0.
- Si calcolano i cammini massimi dal nodo x ad ogni altro nodo, a partire dal primo nodo dell'ordinamento topologico.

$$d(v) = \max_{u|(u,v) \in E} (d(u) + 1)$$

Passi

	0	1	2	3	4	5	6	7
0	0	∞						
1	∞	0						
2	∞	4	0					
3	∞	3		0				
4	∞	6			0			
5	∞	5				0		
6	∞		10				0	

0 → 2 → 6 → 3 → 5 → 1 → 4



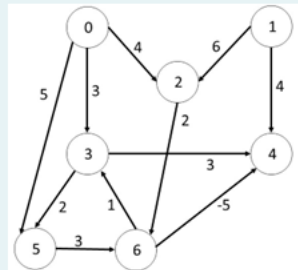
0-2 0-3 0-5 1-2 1-4 2-6 3-4 3-5 5-6 6-3 6-4

Domanda 7

Punteggio max.:
2,00

Contrassegna
domanda

Si determinino per il seguente grafo orientato pesato mediante l'algoritmo di Bellman-Ford i valori di tutti i cammini minimi che collegano il vertice 1 con ogni altro vertice:



Si trattino, qualora necessario, i vertici secondo l'ordine numerico e si assuma che la lista delle adiacenze sia anch'essa ordinata numericamente.

Formato: stime della distanza minima ad ogni passo (inf per ∞)

Passo 0 (inizializzazione)

0: ,1: ,2: ,3: ,4: ,5: ,6:

Passo 1

0: ,1: ,2: ,3: ,4: ,5: ,6:

Passo 2

0: ,1: ,2: ,3: ,4: ,5: ,6:

....

Passo 7

0: ,1: ,2: ,3: ,4: ,5: ,6:

È stato raggiunto un punto fisso? Sì/No. Se Sì, a quale passo? 511 }

Cosa significa raggiungere un punto fisso? In quale contesto? si stabilizza

Breve spiegazione:

Il risultato finale è ottimo? Sì/No si , no CUU rbbgrrr

Breve motivazione:

non hbl.

~~0-2 0-3 0-5~~ 1-2 1-4 2-6 3-4 3-5 5-6 6-3 6-4

~~1 3 5~~ 6 4 7 3 2 3 3 11 8 9

(ordinati)

Algoritmo di Bellman-Ford

1. Si crea una tabella di dimensione $(N + 1) \times N$, con N numero di vertici del grafo. Si utilizzeranno le colonne per definire i passi, mentre le righe si usano per la distanza minima dal nodo scelto fino al nodo indicato nella riga.
2. Si ordinano gli archi in ordine lessicografico e si inizializza la prima colonna della tabella con valore ∞ e 0 sul nodo scelto.
3. Si guarda ogni arco nell'ordine definito. Se è presente un cammino con distanza minore per un determinato nodo, allora si inserisce nella tabella, al determinato nodo, una nuova distanza minima.
4. Si ripete fino a non avere variazioni della distanza minima. Se si hanno variazioni anche al passo $N + 1$, allora il grafo contiene un ciclo a peso negativo, e quindi il cammino minimo perde di senso.

	p_0	p_1	p_2	p_3
0	∞	∞	∞	∞
1	0	0	0	0
2	∞	6	6	6
3	∞	9	9	9
4	∞	3	3	3
5	∞	∞	11	11
6	∞	8	8	9

